Współbieżność I wielowątkowość w Javie

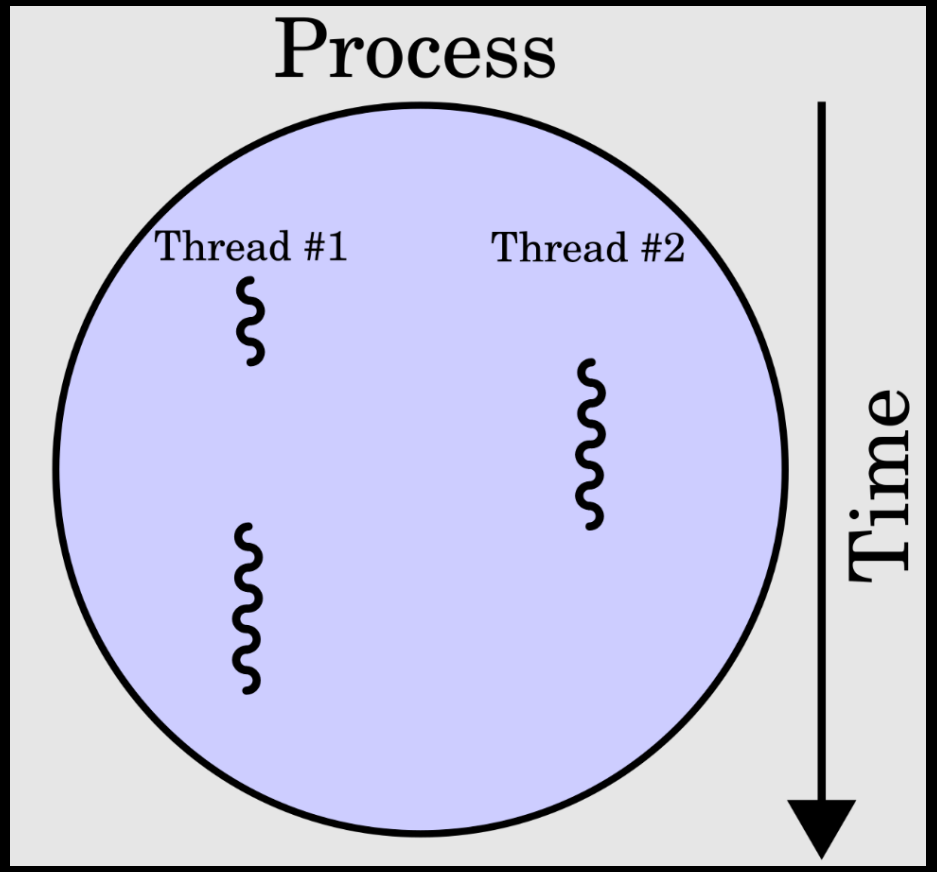
Franciszek Hajduk, Maciej Woźniak

Cel I plan prezentacji:

- Wstęp - Czym jest wątek, wątek a proces
- Współbieżność (concurrency) a równoległość (parallelism) – kluczowe różnice i przykłady zastosowań.
- Cykl życia wątku (tworzenie, uruchamianie, wstrzymywanie, kończenie działania).
- Sposoby tworzenia wątków w Javie: dziedziczenie po klasie Thread vs. implementacja interfejsuRunnable
- Typowe problemy w aplikacjach wielowątkowych: wyścigi (race conditions), zakleszczenia (deadlocks) oraz zagłodzenia (starvation).
- Zarządzanie bezpieczeństwem wątków za pomocą słowa kluczowego synchronized.
- Podstawowe mechanizmy synchronizacji i dostępu do zasobów współdzielonych (np. blokady, semafory).
- Zarządzanie zasobami sprzętowymi przy pomocy puli wątków (Thread Pools) oraz ExecutorService.
- Przegląd i wykorzystanie struktur danych z pakietu java.util.concurrent (np. kolekcje współbieżne).

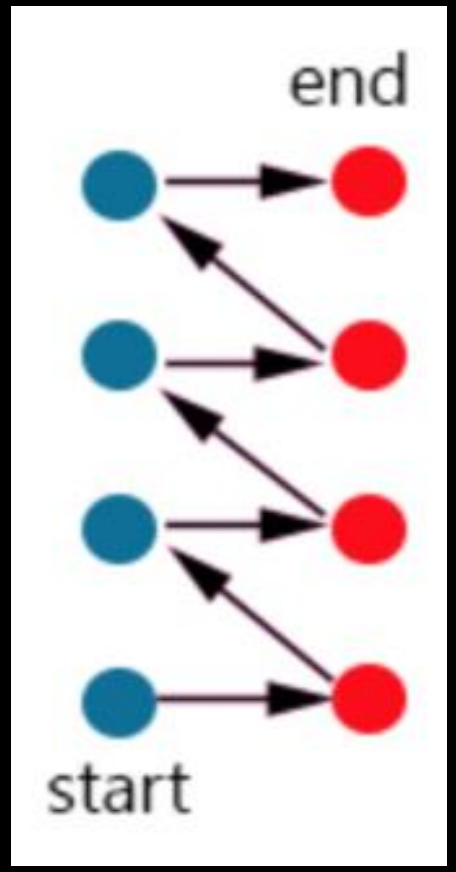
Wstęp - Czym jest wątek?

- Definicja: Najmniejsza jednostka wykonawcza, która może być zarządzana przez system operacyjny
- Wątek a Proces: Proces to uruchomiony program, który ma własną, odizolowaną pamięć.
- Wątek to „ścieżka wykonania” wewnątrz tego procesu. Jeden proces może mieć wiele wątków.
- Współdzielenie zasobów: Wątki wewnątrz jednego procesu dzielą tę samą pamięć (zmienne, obiekty), co pozwala na szybką komunikację, ale rodzi wyzwania.
- Lekkość: Stworzenie wątku jest znacznie „tańsze” dla systemu niż stworzenie całego nowego procesu.



Źródło grafiki: Wikipedia

Współbieżność (Concurrency)



Źródło grafiki: linkedin.com/pulse

- Współbieżność (Concurrency) to cecha struktury programu, pozwalająca na zarządzanie wieloma zadaniami, które startują, działają i kończą się w nakładających się przedziałach czasu.
- W systemach jednordzeniowych realizacja zadań odbywa się poprzez przełączanie kontekstu (Context Switching) – procesor przeskakuje między wątkami, dając iluzję jednoczesności.
- Głównym celem współbieżności jest to, aby jedno zadanie (np. długie pobieranie danych) nie blokowało całego programu (np. interfejsu użytkownika).

Współbieżność - przykład

- Użytkownik 1 żąda danych z serwera.
- Użytkownik 2 przesyła plik.
- Użytkownik 3 pobiera obrazy

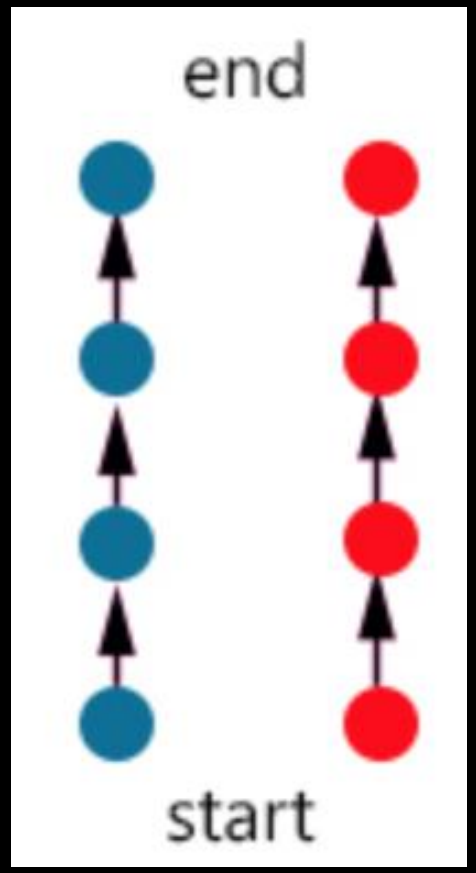
Bez współbieżności każde żądanie musiałoby czekać, aż poprzednie zostanie ukończone

Używając współbieżności:

- Serwer rozpoczyna przetwarzanie żądania Użytkownika 1.
- Oczekując na odpowiedź bazy danych, przechodzi do przesyłania pliku Użytkownika 2.
- Oczekując na przesłanie pliku, rozpoczyna pobieranie obrazów Użytkownika 3.

Procesor przełącza się między tymi zadaniami, aż wszystkie zostaną ukończone.

Równoległość (Parallelism)



Źródło grafiki: linkedin.com/pulse

- Równoległość (Parallelism) to technika polegająca na fizycznym, jednoczesnym wykonywaniu wielu zadań na różnych jednostkach obliczeniowych (np. rdzeniach CPU lub układach GPU).
- W przeciwieństwie do współbieżności, zadania nie przeplatają się – one postępują w czasie dokładnie w tej samej linii.
- Głównym celem równoległości jest skrócenie czasu wykonania ciężkich zadań obliczeniowych (tzw. CPU-bound tasks)

Równoległość - przykład

Są do dyspozycji 3 rdzenie procesora.

Zadanie: Renderowanie 3 klatek animacji 3D

Bez równoległości:

Procesor musi w całości obliczyć klatkę pierwszą, potem drugą, a na końcu trzecią. Całkowity czas to suma czasu trwania wszystkich trzech zadań.

Używając równoległości:

- Rdzeń 1 procesora zajmuje się renderowaniem klatki 1.
- Rdzeń 2 procesora w tym samym ułamku sekundy renderuje klatkę 2.
- Rdzeń 3 procesora jednocześnie przetwarza klatkę 3.

Wszystkie zadania są wykonywane fizycznie w tym samym czasie na różnych jednostkach obliczeniowych, co drastycznie skraca czas operacji.

Gdzie używamy tego w praktyce? (Zastosowania)

- **Serwery i aplikacje webowe:** Obsługa tysięcy użytkowników w tym samym czasie (np. Spring Boot, Apache Tomcat). Każde żądanie HTTP to (zazwyczaj) osobny wątek.
- **Responsywne interfejsy (GUI / Mobile):** Główny wątek rysuje ekrany, a wątki w tle pobierają dane. Zapobiega to "zamarzaniu" aplikacji.
- **Przetwarzanie danych (Big Data / Raportowanie):** Dzielenie gigantycznych kolekcji danych na mniejsze paczki i analizowanie ich równolegle (np. `parallelStream()` w Javie).
- **Zadania w tle (Background Jobs):** Wysyłanie e-maili, generowanie plików PDF, odpytywanie zewnętrznych API bez blokowania głównego procesu.
- **Gry i symulacje:** Osobne wątki dla silnika fizyki, sztucznej inteligencji (AI) i renderowania grafiki.

Cykl życia wątku (Stany)

New - wątek utworzony, ale jeszcze nie uruchomiony (metoda start() nie została jeszcze wywołana)

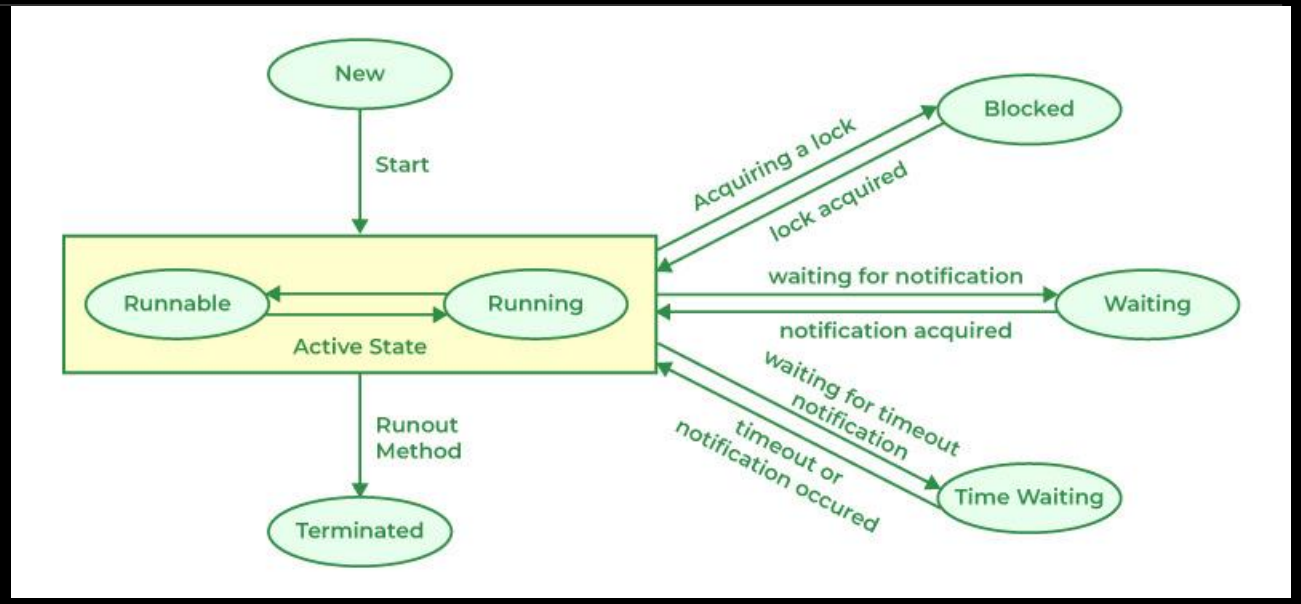
Runnable - wątek jest gotowy do działania i czeka na przydzielenie czasu procesora przez system (tzw. Scheduler)

Running - wątek wykonuje pracę

Blocked - zablokowany (przez inny wątek)

Waiting/Timed_waiting - wątek został uśpiony odpowiednią metodą. Oczekuje na powiadomienie (notify) od innego wątku lub upłynięcie zadanego czasu

Terminated - wątek został zakończony



Źródło grafiki: geeksforgeeks.org

Fundamenty – klasa Thread i interfejs Runnable

- Klasa Thread (java.lang.Thread): Podstawowa klasa w Javie reprezentująca wątek wykonania. Zarządza cyklem życia wątku w maszynie wirtualnej (JVM).
- Interfejs Runnable: Interfejs funkcyjny definiujący zadanie do wykonania. Posiada tylko jedną metodę: public void run(), którą należy zaimplementować - zdefiniować kod, który wykona się po uruchomieniu.
- Konstruktor Thread(Runnable target) pozwala przydzielić zadanie (Runnable) do wątku (Thread).

Kluczowe metody klasy Thread

start()	Tworzy nowy wątek systemowy i uruchamia w nim metodę run(). Nie należy wywoływać run() bezpośrednio
Thread.sleep(ms)	Metoda statyczna, usypia bieżący wątek (ten, który aktualnie wykonuje kod)
t.join()	Powoduje, że bieżący wątek czeka, aż wątek t zakończy pracę.
Thread.yield()	Metoda statyczne, sugestia oddania procesora innym wątkom.
t.interrupt()	Kończy pracę wątku
Thread.currentThread()	Metoda statyczna, wraca referencję do aktualnie bieżącego wątku

Tworzenie wątków w Javie - Dwa podejścia

```java
class MyThread extends Thread {  2 usages
    // Overriding the run method
    @Override
    public void run() {
        System.out.println("This code is running in a thread");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
        System.out.println("This code is outside of the thread");
    }
}
```

Dziedziczenie po klasie Thread

- Brak wielokrotnego dziedziczenia: Java nie pozwala dziedziczyć po więcej niż jednej klasie.
- Ciasne powiązanie (Tight Coupling): wymieszana logika zadania z mechanizmem jego wykonywania.

```java
class MyTask implements Runnable {  2 usages
    // Overriding the run method
    @Override
    public void run() {
        System.out.println("This code is running in a thread");
    }
}

public class Main {
    public static void main(String[] args) {
        MyTask task1 = new MyTask();
        Thread thread1 = new Thread(task1);
        thread1.start();
        System.out.println("This code is outside of the thread");
    }
}
```

Implementacja interfejsu Runnable

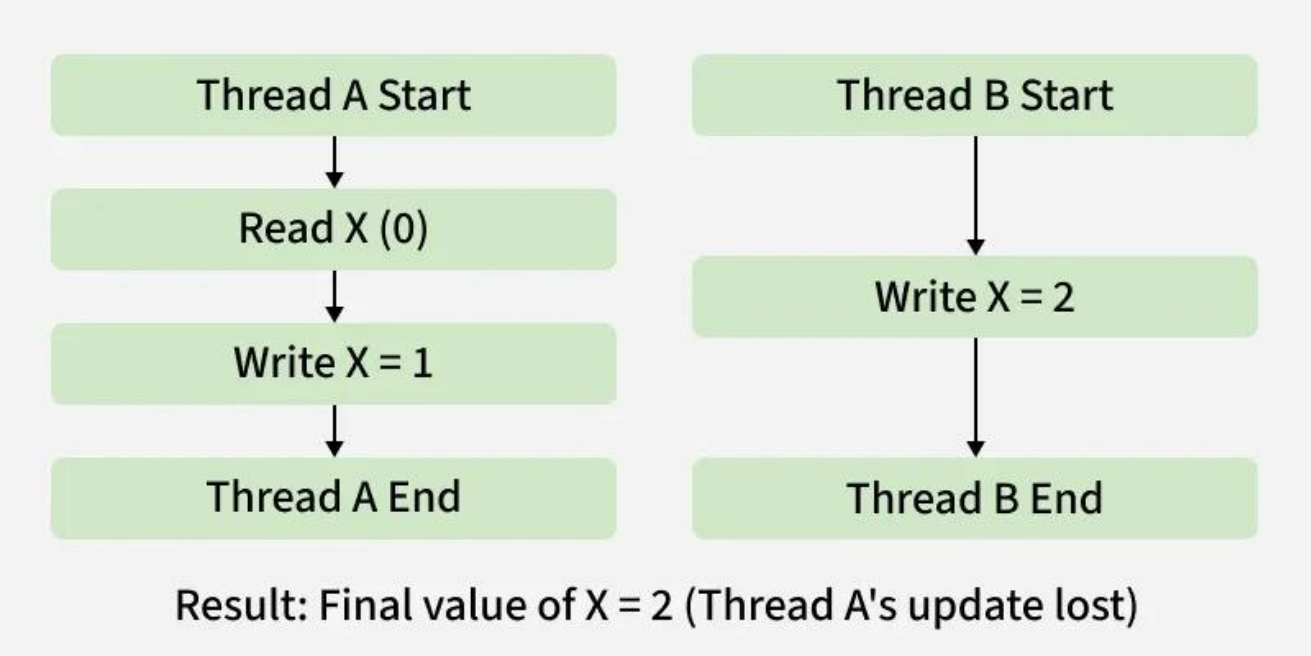
- Elastyczność: Klasa może dziedziczyć po czymkolwiek innym.
- Separacja odpowiedzialności: Oddzielenie zadania (Runnable) od wykonawcy (Thread). To samo zadanie można przekazywać.

Zasoby współdzielone (Wstęp do problemów)

- Co się dzieje, gdy dwa wątki dotykają tych samych danych?
- "Wielowątkowość jest prosta, dopóki wątki ze sobą nie rozmawiają."

Wyścigi (Race Conditions)

Gdy kilka wątków korzysta z tych samych zasobów I próbują je zmienić w tym samym momencie. Wtedy pierwszy lub drugi wątek będzie miał złe dane zmienione przez inny wątek.



Źródło grafiki: geeksforgeeks.org

```java
class Counter {  2 usages
    public int count = 0;  3 usages
    public void increment() {  1 usage
        int temp = count;       // Odczyt
        // Symulacja opóźnienia, aby wymusić przełączenie kontekstu (Race Condition)
        try { Thread.sleep( millis: 1); } catch (InterruptedException e) {}
        count = temp + 1;       // Modyfikacja i Zapis
    }
}               // Metoda increment() nie jest atomowa

public class RaceConditionDemo {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        // Wyrażenie lambda generujące anonimową klasę implementującą interfejs Runnable w locie
        Runnable task = () -> {                     // można użyć wyrażenia lambda, ponieważ
            for (int i = 0; i < 100; i++) {         // Runnable to interfejs funkcyjny
                counter.increment();
            }
        };
        Thread threadA = new Thread(task,  name: "Wątek-A");
        Thread threadB = new Thread(task,  name: "Wątek-B");
        threadA.start();
        threadB.start();
        threadA.join();           // Czekamy na zakończenie obu wątków
        threadB.join();
        System.out.println("Oczekiwany wynik: 200");
        System.out.println("Faktyczny wynik:  " + counter.count);
    }
}
```

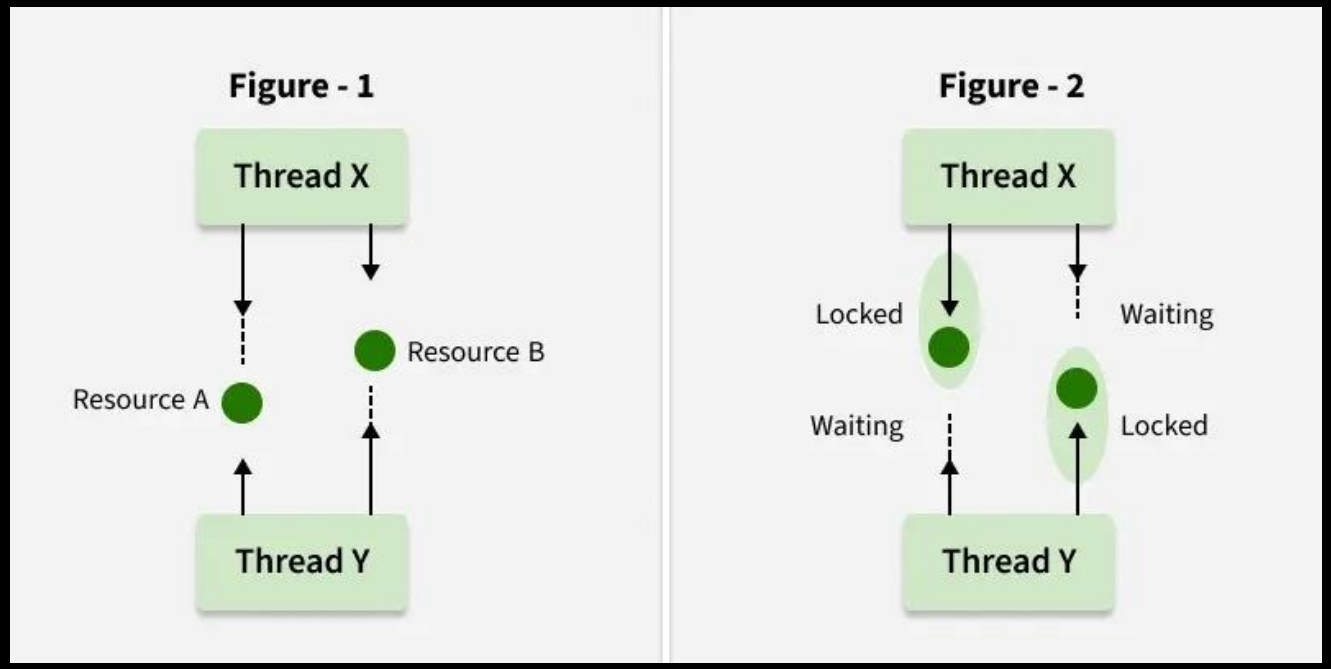
```
Oczekiwany wynik: 200
Faktyczny wynik:  102

Process finished with exit code 0
```

Krok	Wątek-A	Wątek-B	Stan count w pamięci
1	Odczytuje count (10) do swojego temp.	Oczekuje	10
2	Zasypia (Thread.sleep).	Zaczyna działanie	10
3	Śpi	Odczytuje count (10) do swojego temp.	10
4	Śpi	Zasypia (Thread.sleep).	10
5	Budzi się i wykonuje count = 10 + 1.	Śpi	11
6	Kończy iterację	Budzi się i wykonuje count = 10 + 1.	11

Zakleszczenie (Deadlock)

Dzieje się gdy oba wątki są zablokowane przez siebie nawzajem I oba czekają na odpowiedź (uwolnienie) od drugiego



Źródło grafiki: geeksforgeeks.org

```java
public class DeadlockDemo {
    // Dwa zasoby, o które będą "walczyć" wątki
    public static final Object LockA = new Object();  2 usages
    public static final Object LockB = new Object();  2 usages

    public static void main(String[] args) {

        // Wątek 1: Chce najpierw LockA, potem LockB
        Thread thread1 = new Thread(() -> {
            synchronized (LockA) {
                System.out.println("Wątek 1: Trzymam LockA...");

                try { Thread.sleep( millis: 50); } catch (InterruptedException e) {}

                System.out.println("Wątek 1: Czekam na LockB...");
                synchronized (LockB) {
                    System.out.println("Wątek 1: Mam oba locki!");
                }
            }
        }, name: "Wątek-1");

        // Wątek 2: Chce najpierw LockB, potem LockA (odwrotna kolejność!)
        Thread thread2 = new Thread(() -> {
            synchronized (LockB) {
                System.out.println("Wątek 2: Trzymam LockB...");

                try { Thread.sleep( millis: 50); } catch (InterruptedException e) {}

                System.out.println("Wątek 2: Czekam na LockA...");
                synchronized (LockA) {
                    System.out.println("Wątek 2: Mam oba locki!");
                }
            }
        }, name: "Wątek-2");
        thread1.start();
        thread2.start();
    }
}
```

```
Wątek 1: Trzymam LockA...
Wątek 2: Trzymam LockB...
Wątek 2: Czekam na LockA...
Wątek 1: Czekam na LockB...
```

Krok	Wątek 1 (chce A → B)	Wątek 2 (chce B → A)	Stan blokad (Locks)
1	Zajmuje LockA (wykonuje synchronized)	Startuje...	LockA: W1, LockB: wolny
2	Wypisuje: "Trzymam LockA"	Zajmuje LockB (wykonuje synchronized)	LockA: W1, LockB: W2
3	Zasypia (Thread.sleep(50))	Wypisuje: "Trzymam LockB"	LockA: W1, LockB: W2
4	Śpi...	Zasypia (Thread.sleep(50))	LockA: W1, LockB: W2
5	Budzi się i próbuje wziąć LockB	Śpi...	LockA: W1, LockB: W2
6	CZEKA (LockB należy do W2)	Budzi się i próbuje wziąć LockA	LockA: W1, LockB: W2
7	CZEKA (LockB nadal u W2)	CZEKA (LockA należy do W1)	DEADLOCK

Zagłodzenie (Starvation)

Sytuacja, w której wątek nie może uzyskać regularnego dostępu do współdzielonych zasobów i nie może czynić postępów w pracy.

Przyczyny:

- Priorytety wątków: Wątki o wysokim priorytecie dostają więcej czasu procesora
- Niesprawiedliwe blokady: Używanie mechanizmów, które nie gwarantują kolejności dostępu
- Zasobożerne wątki: Jeden lub kilka wątków przetrzymuje blokadę przez bardzo długi czas, nie dając szansy innym.

Skutek: Aplikacja działa, ale wybrane funkcjonalności (obsługiwane przez "głodne" wątki) wydają się zawieszone

```java
public class StarvationDemo {
    private static final Object lock = new Object();  2 usages

    public static void main(String[] args) {
        // Wątek "Chciwy" - ma wysoki priorytet i trzyma blokadę bardzo długo
        Thread greedyThread = new Thread(() -> {
            int workCycles = 0;
            while (true) {
                synchronized (lock) {
                    workCycles++;
                    System.out.println("Wątek VIP: Wykonuję cykl pracy nr " + workCycles);

                    // Symulacja ciężkiej pracy wewnątrz sekcji krytycznej
                    try { Thread.sleep( millis: 100); } catch (InterruptedException e) {}
                }
            }
        }, name: "VIP-Wątek");

        // Wątek "Głodny" - ma niski priorytet i próbuje uzyskać dostęp do tego samego locka
        Thread starvingThread = new Thread(() -> {
            long startTime = System.currentTimeMillis();
            while (true) {
                synchronized (lock) {
                    long waitTime = System.currentTimeMillis() - startTime;
                    System.out.println(">>> GŁODNY WĄTEK:  dostęp po " + waitTime + "ms");

                    startTime = System.currentTimeMillis();
                }
            }
        }, name: "Głodny-Wątek");

        // Ustawienie ekstremalnych priorytetów
        greedyThread.setPriority(Thread.MAX_PRIORITY); // 10
        starvingThread.setPriority(Thread.MIN_PRIORITY); // 1

        System.out.println("symulacja zagłodzenia...");
        starvingThread.start();
        greedyThread.start();
    }
}
```

```
Wątek VIP: Wykonuję cykl pracy nr 785
Wątek VIP: Wykonuję cykl pracy nr 786
Wątek VIP: Wykonuję cykl pracy nr 787
Wątek VIP: Wykonuję cykl pracy nr 788
Wątek VIP: Wykonuję cykl pracy nr 789
Wątek VIP: Wykonuję cykl pracy nr 790
```

Wątek o niskim priorytecie nie jest wykonywany – jest zagłodzony

Rozwiązanie: synchronized

- Mechanizm Monitora: Każdy obiekt w Javie posiada tzw. intrinsic lock (monitor). Słowo synchronized sprawia, że wątek musi "zdobyć klucz" do tego monitora, zanim wejdzie do środka.
- Wzajemne Wykluczanie (Mutex): Gwarantuje, że w danej chwili tylko jeden wątek wykonuje sekcję krytyczną kodu. Inne wątki muszą czekać (stan Blocked).
- Widzialność pamięci: Zapewnia, że zmiany dokonane przez jeden wątek są natychmiast widoczne dla innych wątków po zwolnieniu blokady.

Dwa podejścia:

1. Synchronizacja całej metody Najprostsza, ale najmniej wydajna (blokuje cały obiekt na cały czas trwania metody).

```
public synchronized void increment() {  1 usage
    // Cały kod metody jest chroniony blokadą
    this.count++;
}
```

2. Synchronizacja bloku (Synchronized Block) Zalecana, gdy metoda jest długa, a tylko mały jej fragment modyfikuje współdzielone dane.

```
public void updateData() {  1 usage
    // Logika, która nie wymaga blokady (np. obliczenia)

    synchronized(this) {
        // Sekcja krytyczna - tylko to jest blokowane
        this.count++;
    }
}
```

Zaawansowane blokady – `ReentrantLock`

- Pełna kontrola: Pozwala na ręczne wywoływanie metod lock() oraz unlock(). Nie jest ograniczony do zakresu jednej metody czy bloku klamrowego.
- Elastyczność (tryLock): Możliwość sprawdzenia, czy zasób jest wolny, bez przechodzenia w stan uśpienia (brak blokowania wątku w nieskończoność).
- Obsługa Timeoutów: Możemy spróbować pozyskać blokadę tylko przez określony czas, co jest najskuteczniejszą bronią przeciwko zakleszczeniom (Deadlocks).
- Sprawiedliwość (Fairness): Możliwość ustawienia trybu "fair", w którym wątki są wpuszczane do sekcji krytycznej w kolejności ich przybycia (FIFO), co zapobiega zagłodzeniu.

```java
import java.util.concurrent.locks.Lock;          // Import interfejsu
import java.util.concurrent.locks.ReentrantLock; // Import konkretnej klasy

// W kodzie:
Lock lock = new ReentrantLock();  no usages
```

```java
lock.lock(); // Ręczne zablokowanie
try {
    // Sekcja krytyczna (dostęp do danych)
    count++;
} finally {
    lock.unlock(); // Zawsze zwalniamy blokadę w finally
}
```

Podstawowe użycie

```java
if (lock.tryLock()) {
    try {
        // Robimy coś z zasobem
    } finally {
        lock.unlock();
    }
} else {
    // Zasób zajęty - wykonaj logikę alternatywną
    System.out.println("Nie udało się uzyskać blokady, wracam do innych zadań.");
}
```

Próba pozyskania blokady (tryLock)

- Pokazuje przewagę nad synchronized. Jeśli zasób jest zajęty, program może zrobić coś innego

Semafory (kontrola przepustowości)

- Zasada działania: Semafor zarządza zestawem pozwoleń (permits). Wątek, który chce uzyskać dostęp do zasobu, musi "pobrać" pozwolenie. Jeśli pula jest pusta, wątek czeka, aż inne wątki je zwrócą.
- Kluczowe operacje:
  - acquire() – pobiera pozwolenie (lub blokuje wątek, jeśli brak wolnych).
  - release() – zwraca pozwolenie do puli.
- Zastosowanie: Idealne do ograniczania dostępu do zasobów o limitowanej przepustowości, np. połączeń z bazą danych, liczby drukarek czy liczby osób w "pokoju" (np. serwerze gier).
- Semafor binarny: Specjalny przypadek semafora z 1 pozwoleniem, który działa niemal identycznie jak Mutex (blokada).

```java
import java.util.concurrent.Semaphore;

public class SemaphoreDemo {
    // Tworzymy semafor z 3 pozwoleniami (3 miejsca w czytelni)
    private static final Semaphore readingRoom = new Semaphore( permits: 3);  2 usages

    public static void main(String[] args) {
        for (int i = 1; i <= 6; i++) {
            final int readerId = i;
            new Thread(() -> {
                try {
                    System.out.println("Czytelnik " + readerId + " czeka przed czytelnią...");

                    readingRoom.acquire(); // Pobranie pozwolenia

                    System.out.println(">>> Czytelnik " + readerId + " wszedł i czyta.");
                    Thread.sleep((long) (Math.random() * 3000)); // Symulacja czytania

                    System.out.println("<<< Czytelnik " + readerId + " wychodzi i zwalnia miejsce.");
                } catch (InterruptedException e) {
                    e.printStackTrace();
                } finally {
                    readingRoom.release(); // ZWOLNIENIE pozwolenia - kluczowe
                }
            }).start();
        }
    }
}
```

```
Czytelnik 3 czeka przed czytelnią...
Czytelnik 5 czeka przed czytelnią...
Czytelnik 1 czeka przed czytelnią...
>>> Czytelnik 3 wszedł i czyta.
>>> Czytelnik 5 wszedł i czyta.
Czytelnik 2 czeka przed czytelnią...
Czytelnik 6 czeka przed czytelnią...
Czytelnik 4 czeka przed czytelnią...
>>> Czytelnik 1 wszedł i czyta.
<<< Czytelnik 3 wychodzi i zwalnia miejsce.
>>> Czytelnik 2 wszedł i czyta.
<<< Czytelnik 5 wychodzi i zwalnia miejsce.
>>> Czytelnik 6 wszedł i czyta.
<<< Czytelnik 1 wychodzi i zwalnia miejsce.
>>> Czytelnik 4 wszedł i czyta.
<<< Czytelnik 6 wychodzi i zwalnia miejsce.
<<< Czytelnik 2 wychodzi i zwalnia miejsce.
<<< Czytelnik 4 wychodzi i zwalnia miejsce.

Process finished with exit code 0
```

Pule Wątków I ExecutorService

- Problem z new Thread(): Tworzenie wątku jest kosztowne (wymaga czasu i pamięci). Tworzenie tysięcy wątków w pętli może doprowadzić do wyczerpania zasobów systemu i błędu OutOfMemoryError.
- Rozwiązanie – Pula wątków: Zbiór wcześniej utworzonych, gotowych do pracy wątków. Zamiast tworzyć nowy wątek dla każdego zadania, przekazujemy zadanie do puli, a wolny wątek je podejmuje.
- ExecutorService: Wysokopoziomowy interfejs w Javie, który zarządza pulą wątków i kolejką zadań.
- Zalety:
  - Kontrola zasobów: Ograniczenie maksymalnej liczby wątków (np. do 10).
  - Wydajność: Ponowne wykorzystanie (reuse) wątków oszczędza czas procesora.
  - Zarządzanie: Łatwe zamykanie wątków i monitorowanie ich stanu.

Rodzaje pul wątków i kluczowe metody

newFixedThreadPool(n)	Stała liczba wątków. Najlepsza, gdy znane są możliwości sprzętu.
newCachedThreadPool()	Tworzy nowe wątki w razie potrzeby, a nieużywane usuwa. Dobra dla wielu krótkich zadań.
newSingleThreadExecutor()	Tylko jeden wątek. Gwarantuje, że zadania wykonają się po kolei (FIFO).
newScheduledThreadPool(n)	Pozwala planować zadania (np. "uruchom to za 5 sekund" lub "uruchamiaj co minutę").

execute(Runnable)	Metoda typu fire-and-forget. Nie zwraca żadnego wyniku.
shutdown()	Inicjuje procedurę zamknięcia. Pula przestaje przyjmować nowe zadania, ale kończy te już zlecone.
shutdownNow()	Próbuje natychmiast zatrzymać wszystkie aktywne zadania i czyści kolejkę czekających.
awaitTermination(time, unit)	Blokuje wątek główny, czekając na fizyczne zakończenie pracy wszystkich wątków w puli.

```
ExecutorService executor = Executors.newFixedThreadPool( nThreads: 10);
```

Przykładowe utworzenie puli wątków

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class PerformanceComparison {
    private static final int TASKS_COUNT = 1000;  3 usages

    public static void main(String[] args) throws InterruptedException {
        Runnable task = () -> {
            double result = Math.tan(Math.atan(Math.tan(Math.atan(1.0))));      // Symulacja krótkiej pracy
        };

        long startManual = System.currentTimeMillis();             // TEST 1: ręczne tworzenie wątków (new Thread)
        Thread[] threads = new Thread[TASKS_COUNT];

        for (int i = 0; i < TASKS_COUNT; i++) {
            threads[i] = new Thread(task);
            threads[i].start();
        }
        for (Thread t : threads) {
            t.join(); // Czekamy na każdy wątek
        }
        long endManual = System.currentTimeMillis();
        System.out.println("Czas (new Thread): " + (endManual - startManual) + " ms");

        long startPool = System.currentTimeMillis();                // TEST 2: użycie Puli Wątków (ThreadPool)
        ExecutorService executor = Executors.newFixedThreadPool( nThreads: 10);    //pula wątków do obsłużenia 1000 zadań

        for (int i = 0; i < TASKS_COUNT; i++) {
            executor.execute(task);
        }
        executor.shutdown();
        executor.awaitTermination( timeout: 1, TimeUnit.MINUTES);

        long endPool = System.currentTimeMillis();
        System.out.println("Czas (ThreadPool): " + (endPool - startPool) + " ms");
    }
}
```

```
Czas (new Thread): 146 ms
Czas (ThreadPool): 7 ms

Process finished with exit code 0
```

ExecutorService zarządza wątkami, optymalizuje pracę poprzez ich ponowne wykorzystanie, eliminując ogromny narzut czasowy i pamięciowy związany z ich ciągłym tworzeniem.

Współbieżne struktury danych (java.util.concurrent)

Kolekcje współbieżne to specjalne kontenery na dane, które pozwalają uniknąć ręcznego używania synchronized czy Lock przy każdym odczycie i zapisie.

- Problem: Standardowe kolekcje (ArrayList, HashMap) nie są bezpieczne wątkowo. Ich modyfikacja przez wiele wątków naraz prowadzi do uszkodzenia danych lub wyjątku ConcurrentModificationException
- Synchronizowane kolekcje: Collections.synchronizedMap() blokuje całą strukturę przy każdym dostępie, co drastycznie obniża wydajność w systemach wielowątkowych.

Struktura	Odpowiednik "zwykły"	Zasada działania	Kiedy stosować?
ConcurrentHashMap	HashMap	Blokuje tylko fragmenty mapy przy zapisie. Odczyt jest zazwyczaj bezblokowy.	Gdy wiele wątków intensywnie czyta i zapisuje dane w mapie (np. wspólny cache).
CopyOnWriteArrayList	ArrayList	Każda modyfikacja tworzy nową kopię tablicy. Odczyty działają na starej wersji bez żadnych blokad.	Gdy odczyty są bardzo częste, a zapisy zdarzają się rzadko (np. lista słuchaczy zdarzeń / Listeners).
ArrayBlockingQueue	ArrayList (jako kolejka)	Kolejka o stałym rozmiarze. Blokuje wątek przy próbie zapisu do pełnej lub czytania z pustej kolejki.	Idealna do wzorca Producent-Konsument przy ograniczonej pamięci.
LinkedBlockingQueue	LinkedList (jako kolejka)	Działa podobnie do powyższej, ale zazwyczaj ma opcjonalnie nieograniczony rozmiar.	Gdy tempo produkcji zadań może chwilowo znacznie przewyższać tempo konsumpcji.
ConcurrentLinkedQueue	LinkedList (jako kolejka)	Kolejka bezblokowa (Lock-free), oparta na algorytmach CAS (Compare-And-Swap).	Tam, gdzie priorytetem jest najwyższa wydajność i brak usypiania wątków.
ConcurrentSkipListMap	TreeMap	Współbieżna mapa, która utrzymuje klucze w porządku sortowania.	Gdy potrzebujesz bezpiecznej wątkowo mapy, która jest zawsze posortowana.

```java
import java.util.*;
import java.util.concurrent.*;

public class ConcurrentCollectionsDemo {
    public static void main(String[] args) throws InterruptedException {
        Map<String, Integer> map = new ConcurrentHashMap<>();
        //Map<String, Integer> map = new HashMap<>();


        ExecutorService executor = Executors.newFixedThreadPool( nThreads: 10);

        for (int i = 0; i < 1000; i++) {
            int finalI = i;
            executor.execute(() -> {
                map.put("Klucz-" + finalI, finalI);
            });
        }

        executor.shutdown();
        executor.awaitTermination( timeout: 1, TimeUnit.MINUTES);

        System.out.println("Rozmiar mapy: " + map.size()); // Zawsze 1000 przy ConcurrentHashMap
    }
}
```

```
Rozmiar mapy: 1000

Process finished with exit code 0
```

```java
//Map<String, Integer> map = new ConcurrentHashMap<>();
Map<String, Integer> map = new HashMap<>();
```

```
Rozmiar mapy: 995

Process finished with exit code 0
```

- Standardowy HashMap (wynik 995/1000): Nie posiada wewnętrznych mechanizmów synchronizacji. Gdy wiele wątków próbuje zapisać dane w tym samym momencie, dochodzi do wyścigów (Race Conditions).
- ConcurrentHashMap (wynik 1000/1000): Gwarantuje spójność danych dzięki wewnętrznemu blokowaniu fragmentów mapy.
- Wniosek: Użycie niewłaściwej struktury danych w środowisku wielowątkowym prowadzi do błędów, które są trudne do wykrycia, ponieważ występują losowo (wynik 995 może być inny przy każdym uruchomieniu).

Dziękujemy za uwagę

Bibliografia

- Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). Java Concurrency in Practice. Addison-Wesley Professional
- Bloch, J. (2018). Effective Java (3rd Edition). Addison-Wesley Professional. (Rozdział 11)
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th Edition). Wiley. (Rozdział 4)
- Oracle. The Java Tutorials: Concurrency. Pobrano z: https://docs.oracle.com/javase/tutorial/essential/concurrency/
- MIT OpenCourseWare. (2016). Software Construction - Concurrency. Massachusetts Institute of Technology. Pobrano z: https://web.mit.edu/6.005/www/fa16/